이 페이지

1. 1. Efficiency — 계산 효율성 (9/7)
2. 2. Recursion — 재귀와 분할 정복 (9/9 · 9/14)
3. 3. Graph — 탐색과 최단 경로 (9/16 · 9/21 · 9/23)
4. 4. Greedy — MST · 스케줄링 · Huffman (9/28 · 9/30 · 10/5)
5. 5. Dynamic Programming (10/7)

한 장 요약 — 중간고사 전체 범위

2026-10-17 기준 · 문제풀이용 핵심만 · 예시와 추적은 접어 둠 (▸ 눌러서 확인) · 자세한 설명은 각 장 노트로

1. Efficiency — 계산 효율성 (9/7)

알고리즘을 세 질문으로 평가	**맞는가 / 시간 $T(n)$ / 더 빠르게 되는가**. 풀이 순서: problem formulation → 설계 → 정확성 → 시간 분석
시간의 기준	기준은 초가 아니라 입력 크기 n당 기본 연산 횟수 (1회 = $O(1)$). $T(n)$ = worst case 최대 횟수
polynomial time	$O(n^d)$ (d 상수)를 보통 efficient라 부름

알고리즘	아이디어	시간
FIB1 (정의 재귀)	$F_n = F_{n-1} + F_{n-2}\ F_0 = 0,\ F_1 = 1$ 그대로	$\Theta(\varphi^n), \varphi = 1.618$ (호출 $2F_{n+1} - 1$번)
FIB2 (배열 저장)	f[0..n]에 저장	$O(n)$ 시간·공간
Convex Hull — Brute force	모든 쌍 (p, q), 왼쪽 판정 D의 부호 $O(1)$	$O(n^3)$
Convex Hull — Graham's scan	x 정렬 후 세 점 p, q, r 검사 — r이 오른쪽이면 앞으로, 왼쪽이면 q **삭제**	$O(n \log n)$ (정렬이 지배)

점근 표기 — 극한 규칙 (f ≥ 0, g(n) > 0이고 극한이 존재할 때)

$$\lim f/g \neq \infty \Rightarrow O \quad \neq 0 \Rightarrow \Omega \quad \neq 0, \infty \Rightarrow \Theta \quad = 0 \Rightarrow o \quad = \infty \Rightarrow \omega$$

Transitivity	$f = O(g), g = O(h) \Rightarrow f = O(h)$
Additivity	$f, g = O(h) \Rightarrow f + g = O(h)$
다항식	$a_0 + \cdots + a_d n^d = \Theta(n^d)$ $(a_d > 0)$
성장 순서 (n이 클 때만)	$\log n < n < n \log n < n^2 < n^3 < 1.5^n < 2^n < n!$. 로그 밑 무관, $\log n = O(n^x), n^d = O(r^n)$ $(r > 1)$

예시: 극한 규칙으로 점근 비교

f	g	극한	결론
$14n^2$	n^2	14	Θ
$n \log^2 n$	n^2	0	o
3^n	n^5	∞	ω (역전은 n ≥ 11)
$\log_2 n$	$\log_{10} n$	3.32	Θ (로그 밑은 상수배)

자주 틀리는 것

FIB1	$O(2^n)$은 맞지만 **$\Theta(2^n)$은 아님** (밑 $\varphi < 2$)
성장 순서	n이 클 때만 — 작은 n에선 $n^5 > 3^n$
Graham 삭제	단계 2에서 지우는 것은 q (r 아님)

2. Recursion — 재귀와 분할 정복 (9/9 · 9/14)

reduction	문제 A를 B의 알고리즘(블랙박스)으로 풂 — ① 입력 변환 ② B 실행 ③ 해 변환. $T(A) = T(B) +$ (① + ③) — 단, T(B)는 변환 후 입력 크기 n′ 기준
recursion	A와 B가 같은 문제인 reduction (B의 인스턴스가 더 작음). 설계 3가지: 작은 인스턴스 만드는 법 / base case / 작은 해 → 원래 해 (정확성은 귀납법)

알고리즘	점화식	시간
Tower of Hanoi	$T(n) = 2T(n-1) + O(1)$	$2^n - 1$ 이동
binary search	$T(n) = T(\lceil n/2 \rceil) + O(1)$	$O(\log n)$ (최악 비교 $\lfloor \log_2 n \rfloor + 1$번)
mergesort	$T(n) = 2T(n/2) + O(n)$	$O(n \log n)$ (최선·최악 동일)
quicksort	$T(n) = T(r-1) + T(n-r) + O(n)$	최선·평균(입력 무작위 가정) $O(n \log n)$, 최악 $O(n^2)$ (정렬된 입력 + 끝 pivot)
정수 곱셈 4번 호출	$T(n) = 4T(n/2) + O(n)$	$O(n^2)$
Karatsuba	$T(n) = 3T(n/2) + O(n)$	$O(n^{\log_2 3}) \approx O(n^{1.585})$
Strassen	$T(n) = 7T(n/2) + O(n^2)$	$O(n^{\log_2 7}) \approx O(n^{2.81})$
selection (무작위 pivot)	E[T(n)] ≤ E[T(3n/4)] + O(n)	기대 $O(n)$, 최악 $O(n^2)$
median of medians	$T(n) \le O(n) + T(n/5) + T(7n/10)$	최악에도 $O(n)$ (합 $9n/10 < n$)
closest pair	$T(n) \le 2T(n/2) + O(n)$	$O(n \log n)$ (띠에서 뒤 11개만 비교 — 보수적 상한, 표준 packing 기준 최대 7개)

좋은 pivot은 순위가 25~75 백분위에 해당하는 pivot이다. 무작위로 pivot을 선택하면 좋은 pivot이 나올 확률은 적어도 1/2이다. 좋은 pivot이 나올 때까지 다시 고르면 기대 시도 횟수는 최대 2회이며, 각 시도의 비용은 O(n)이다. 좋은 pivot을 선택하면 재귀할 부분문제의 크기는 최대 3n/4이다. 따라서 기대 시간의 점화식은 E[T(n)] ≤ E[T(3n/4)] + O(n)이다.

master theorem — $T(n) = aT(\lceil n/b \rceil) + O(n^d)$

$$T(n) = \begin{cases} O(n^d) & d > \log_b a \ (\text{decreasing}) \\ O(n^d \log n) & d = \log_b a \ (\text{equal}) \\ O(n^{\log_b a}) & d < \log_b a \ (\text{increasing}) \end{cases}$$

a · b · d	a = 호출 수, b = 크기 줄이는 비율, d = 합치는 비용 차수

적용 불가	$T(n-1)$ **꼴(Hanoi)은 불가 → 펼치기** → 크기가 다른 subproblem은 재귀 트리

Karatsuba 핵심	$x_Ly_R + x_Ry_L = (x_L + x_R)(y_L + y_R) - x_Ly_L - x_Ry_R$ → 곱셈 3번. $xy = P_1 \cdot 2^n + (P_3 - P_1 - P_2) \cdot 2^{n/2} + P_2$
Strassen	4블록 분할만으론 곱 8번 = $O(n^3)$ (이득 없음) → 7번으로 줄여야 함
median of medians	5개씩 블록 → 블록 중앙값의 중앙값 M → 양쪽 모두 $\geq 3n/10$ 제거. 블록 크기 3이면 합이 n 그대로라 $O(n \log n)$ 5개씩 블록으로 나누어 각 블록의 중앙값을 구하고, 그 중앙값들의 중앙값 M을 피벗으로 선택한다. M을 기준으로 양쪽에서 각각 일정 비율 이상의 원소를 제거할 수 있다. 블록 크기가 3이면 재귀 호출들의 크기 합이 원래 입력 크기와 같아져 선형 시간 보장이 성립하지 않는다.
closest pair	띠 폭 2δ, 칸 변 $\delta/2$ → 4열 × 3행 = 12칸 → 칸당 점 1개 이하

자주 틀리는 것

결론의 $\Theta(\cdot)$는 입력 크기 n이 커질 때 해당 함수가 상수배 범위 안에서 위아래로 동시에 묶이는 점근적 타이트 바운드(tight bound)를 의미한다.

Hanoi 둘째 호출	Hanoi(n−1, **B, C, A**) — 출발이 B, spare가 A
master theorem	$T(n/b)$ 꼴 균등 분할만
selection	셋째 경우는 $k - (\lvert S_L\rvert + \lvert S_v\rvert)$로 보정

3. Graph — 탐색과 최단 경로 (9/16 · 9/21 · 9/23)

항목	인접 행렬	인접 리스트
공간	$O(n^2)$	$O(n+m)$
간선 확인	$O(1)$	$O(\min(\deg u, \deg v))$
모든 간선	$\Theta(n^2)$	$\Theta(n+m)$
적합	dense	sparse

tree Theorem + 탐색

동치 조건	n개 노드: connected / acyclic / 간선 $n-1$개 중 **둘이면 나머지 하나**
무방향	connected: 간선 $= n-1$ tree, $> n-1$ cycle, $< n-1$ 연결 불가
방향	**간선 수로 판단 불가 → DFS back edge**

DFS	재귀(stack), $O(\|V\|+\|E\|)$ (각 간선 2번 검사). pre/post = explore 시작·끝. 무방향 DFS엔 cross 간선 없음
BFS	큐(FIFO), $O(\|V\|+\|E\|)$. 층별 탐색, 같은 층 간선 → 홀수 cycle (무방향). 간선 길이 1이면 dist = 최단 거리

간선	뜻	pre/post 판정
tree	DFS가 따라가 발견	$x_{pre} < y_{pre} < y_{post} < x_{post}$
forward	조상 → 자손 (tree 아님)	tree와 같음 (구분 불가)
back	자손 → 조상	$y_{pre} < x_{pre} < x_{post} < y_{post}$
cross	서로 무관	$y_{post} < x_{pre}$

cycle 판정	**cycle ⇔ DFS가 back edge를 찾음 (방향·무방향 공통)** ($O(\|V\|+\|E\|)$, 모든 정점에서 시작하는 DFS forest 전체를 기준으로 판정). topological ordering = DAG를 post 내림차순으로 나열 = source 제거 반복 (in-degree 카운터, $O(\|V\|+\|E\|)$). DAG는 source·sink 각 1개 이상

최단 경로 알고리즘

Dijkstra	간선 가중치 ≥ 0만 (음수 불가). deletemin으로 꺼낸 노드 확정 + 나가는 간선 update(decreasekey). $O((V+E)\log V)$ (binary heap). array $O(V^2)$, Fibonacci $O(V\log V+E)$. dense면 array, sparse면 heap
Bellman-Ford	음수 가능. 모든 간선을 $\|V\|-1$번 update, $O(VE)$. 간선 i개 경로는 라운드 i에 final. $\|V\|$번째 라운드에도 줄면 출발점에서 도달 가능한 negative cycle

DAG	선형화 순서로 나가는 간선 update, $O(\|V\| + \|E\|)$, 음수 OK. 부등호 뒤집으면 longest path

예시: Dijkstra 추적 (슬라이드 22)

deletemin	decreasekey 뒤	A B C D E
시작	dist(A) = 0	0 ∞ ∞ ∞ ∞
A (0)	B=4, C=2	0 4 2 ∞ ∞
C (2)	B=3, D=6, E=7	0 3 2 6 7
B (3)	D=5, E=6	0 3 2 5 6
D (5)	없음	0 3 2 5 6
E (6)	E→D: 7 > 5 그대로	0 3 2 5 6

prev: B←C, C←A, D←B, E←B → A에서 E까지 A→C→B→E (2+1+3 = 6)

DP로 푸는 최단 경로

Floyd-Warshall:

$$\text{dist}(i,j,k) = \min\{\text{dist}(i,k,k-1) + \text{dist}(k,j,k-1),\ \text{dist}(i,j,k-1)\}, \quad O(n^3)$$

기저 dist(i, j, 0)은 i = j이면 0, 간선 (i, j)가 있으면 그 길이, 없으면 ∞이다.

TSP:

$$C(S,j) = \min_{i \in S,\, i \neq j} \{C(S-\{j\}, i) + d_{ij}\}, \quad O(n^2 2^n)$$

기저 C({1, j}, j) = d(1, j)이다. 최종 답은 출발점 1로 돌아오는 비용까지 포함하여 $\min_{j \neq 1}\{C(V,j) + d(j,1)\}$이다.

all-pairs를 단일 출발 n번으로: BFS·DAG $O(V^3)$, Dijkstra $O(V^3 \log V)$, Bellman-Ford $O(V^4)$

그래프가 연결되어 있으면 $E \geq n-1$이므로, 단일 출발점 알고리즘을 모든 n개 출발점에서 반복하는 전체 시간 복잡도는 BFS/DAG의 경우 $O(nE)$, binary-heap Dijkstra의 경우 $O(nE \log n)$, Bellman-Ford의 경우 $O(n^2 E)$이다. 연결성이 보장되지 않으면 $E = \Omega(n)$을 가정할 수 없으므로, 정점 초기화 비용을 포함해 각각 $O(n(n+E))$, $O(n(n+E)\log n)$, $O(n^2(E+1))$로 표기한다.

자주 틀리는 것

tree/cycle 판단	간선 수($n-1$)로 tree/cycle 판단은 **무방향만** — 방향은 DFS back edge
Dijkstra	음수 간선에서 틀림 → Bellman-Ford
DFS tree 깊이	최단 거리가 아님 → BFS

4. Greedy — MST · 스케줄링 · Huffman (9/28 · 9/30 · 10/5)

greedy	매 단계 가장 이득인 조각을 고름. 어려운 것은 **최적성 증명**과 시간 분석. 증명 도구: induction, cut property, 교환 논증

문제	규칙	시간
Selecting Breakpoints	x에서 $x+C$ 이내 가장 먼 충전소. $b_p = x$면 no solution	$O(n)$ (정렬 필요 시 $+O(n \log n)$)

Kruskal (MST)	가중치 순으로 find(u) ≠ find(v)면 추가 + union	$O(\|E\| \log \|V\|)$
Prim (MST)	subtree를 S–$V \setminus S$ 최소 간선으로 확장 (cost 갱신 + decreasekey)	$O(\|E\| \log \|V\|)$
Interval Scheduling	earliest finish time 순, compatible하면 선택	$O(n \log n)$
Interval Partitioning	시작 시각 순, compatible한 강의실 or 새 강의실 → 강의실 수 = depth	$O(n \log n)$
Huffman	가장 낮은 두 빈도를 합쳐 새 노드, $n-1$번 반복	$O(n \log n)$
Clustering (max spacing)	Kruskal을 연결 성분 k개에서 멈춤 = MST의 비싼 $k-1$개 간선 삭제	spacing = $(k-1)$번째로 비싼 MST 간선의 길이

cut property + union-find

> X가 어떤 MST의 일부이고 cut $(S, V \setminus S)$를 X가 가로지르지 않으면, cut의 **가장 가벼운 간선** e **에 대해** $X \cup \{e\}$**도 어떤 MST의 일부**

union by rank	낮은 rank를 높은 root 아래에. rank k인 root는 노드 $\geq 2^k$개 → 높이 $\leq \log n$. rank +1은 rank가 같을 때만

interval scheduling 반례	earliest start: L[0,9) 하나 vs 4개 짧은 것. shortest interval: M[4,7) 하나 vs A[0,5), B[6,11)
Huffman 최적성	가장 낮은 두 빈도는 최하단 형제. cost = 루트를 제외한 내부 노드의 빈도 합 + $\sum f_i$, $\text{cost}(T) = \text{cost}(T') + f_1 + f_2$

예시: Kruskal 추적

간선 B–C 1, C–D 2, B–D 2, C–F 3, A–D 4, D–F 4, E–F 4, A–B 5, C–E 5, A–C 6

① B–C 1 추가 / C–D 2 추가 / B–D 2 버림 (이미 한 집합) / C–F 3 추가 / A–D 4 추가 / D–F 4 버림 / E–F 4 추가 (집합 1개, 합 14) / 나머지 버림

② B–D 2와 C–D 2 중 하나만 고르면 되므로 MST는 2개 (둘 다 합 14)

자주 틀리는 것

Kruskal이 버리는 간선	두 끝점의 find가 **같은** 것
union by rank	rank +1은 두 root의 rank가 같을 때만
interval scheduling	earliest start, shortest interval, fewest conflicts는 반례가 있음

5. Dynamic Programming (10/7)

DP	recursion without repetition. ① 문제를 recursive하게 정의 (부분문제 + 점화식 — 어려운 쪽) ② 작은 것부터 table에 저장. memoization (top-down) vs table (bottom-up) DP는 부분문제 사이의 점화식을 세우고, 중복되는 상태를 memoization 또는 bottom-up table로 한 번씩 계산하는 방법이다. 부분문제 정의와 점화식 설계가 핵심이며, base case와 계산 순서를 명시한다.

1차원 DP

bit strings (1이 연속 두 번 X):

$$S(i) = S(i-1) + S(i-2),\ S(0) = 1,\ S(1) = 2 \Rightarrow S(i) = F_{i+2}$$

largest sum subarray:

$$S(i) = \max\{S(i-1) + A[i],\ A[i]\}, \quad \text{답} = \max_i S(i), \quad O(n)$$

기저 S(1) = A[1]이다.

LIS:

$$\mathrm{LIS}(i) = 1 + \max\{\mathrm{LIS}(j) \mid j < i,\ A[j] < A[i]\}, \quad O(n^2)$$

$L(i)$(= LIS(i))를 i번째 원소에서 끝나는 최장 증가 부분수열의 길이라고 정의한다. 이전에 더 작은 원소가 없으면 $L(i) = 1$이며, 최종 답은 $\max_{1 \le i \le n} L(i)$에서 읽는다. 실제 부분수열은 각 상태의 최적 선택을 기록하고 역추적하여 복원한다. 이는 모든 가능한 끝점 중 가장 긴 증가 부분수열의 길이이다.

weighted interval:

$$\mathrm{OPT}(j) = \max\{v_j + \mathrm{OPT}(p(j)),\ \mathrm{OPT}(j-1)\}, \quad O(n \log n)$$

기저 조건은 $OPT(0) = 0$이다. 모든 구간을 고려한 최종 최적 가치는 $OPT(n)$에서 읽으며, 선택한 구간은 각 단계에서 $OPT(j-1)$을 선택했는지, $v_j + OPT(p(j))$를 선택했는지 역추적하여 복원한다. 먼저 구간을 종료 시각의 오름차순으로 정렬하고, 정렬된 순서대로 인덱스 1,⋯,n을 붙인다. p(j)는 구간 j와 겹치지 않으면서 j보다 앞에 있는 가장 큰 인덱스이다. 그런 compatible한 구간이 없으면 p(j)=0으로 둔다.

segmented least squares:

$$\mathrm{OPT}(j) = \min_{1 \le i \le j} \{e(i,j) + c + \mathrm{OPT}(i-1)\}, \quad O(n^2)$$

기저 조건은 $OPT(0) = 0$이다. 구간 (i, j)의 제곱 오차 $e(i, j)$는 누적합으로 계산하며, 최종 최소 비용은 $OPT(n)$에서 읽는다. 최적 분할은 각 상태에서 최솟값을 만드는 마지막 구간의 시작점 i를 기록한 뒤, $i-1$ 상태로 역추적하여 복원한다. 여기서 e(i, j)는 점 i부터 j까지를 하나의 직선으로 근사할 때의 최소 제곱 오차이고, c는 구간 하나를 추가할 때 드는 고정 비용이다.

누적합 $\sum x, \sum y, \sum x^2, \sum xy, \sum y^2$으로 e(i,j)를 $O(1)$에 계산

트리 · 2차원 · 구간 DP

트리 독립 집합 (IS(u) = u를 루트로 한 서브트리의 최대 독립 집합 크기):

$$\mathrm{IS}(u) = \max\left\{ \sum_{v \in \text{children}} \mathrm{IS}(v),\ 1 + \sum_{w \in \text{grandchildren}} \mathrm{IS}(w) \right\}, \quad O(n)$$

edit distance:

$$E(i,j) = \min\{E(i-1,j)+1,\ E(i,j-1)+1,\ E(i-1,j-1)+\mathrm{diff}(i,j)\}, \quad O(mn)$$

E(i, j)는 첫째 문자열의 처음 i글자와 둘째 문자열의 처음 j글자 사이의 편집 거리이다. 편집 거리(edit distance)에서 빈 문자열과 길이 i인 문자열 사이의 거리는 E(i,0)=i이고, 길이 j인 문자열과 빈 문자열 사이의 거리는 E(0,j)=j이다. 삽입(insertion)과 삭제(deletion)의 비용은 각각 1이며, 대체(substitution)의 비용은 두 문자가 같으면 0, 다르면 1이다.

subset sum:

$$\mathrm{SS}(i,t) = \mathrm{SS}(i-1,t) \vee \mathrm{SS}(i-1,t-X[i]), \quad O(nT)$$

$$SS(0,0) = \text{true},\ SS(0,t>0) = \text{false},\ t - X[i] < 0 \Rightarrow \text{false}$$

knapsack unlimited:

$$K(w) = \max_{w_i \le w}\{K(w-w_i)+v_i\}$$

0/1:

$$K(w,j) = \max\{K(w,j-1),\ K(w-w_j,j-1)+v_j\}, \quad O(Wn)$$

무제한 배낭(Unbounded Knapsack): 물건 i의 무게를 $w_i > 0$, 가치를 v_i, 배낭의 용량을 w라고 하자. 각 물건을 원하는 횟수만큼 선택할 수 있을 때, $K(w)$를 총무게가 w 이하인 선택의 최대 가치라고 정의한다. 기저 조건은 $K(0) = 0$이다. 용량 w에 대한 점화식은 $K(w) = \max(\{0\} \cup \{K(w-w_i)+v_i \mid w_i \le w\})$이다. 즉, 용량에 들어갈 수 있는 물건만 후보로 고려하며, 들어갈 수 있는 물건이 하나도 없으면 아무것도 선택하지 않아 $K(w) = 0$이다. 각 용량 $w = 1, \ldots, W$를 작은 값부터 계산하고, 각 용량에서 모든 물건을 확인하면 시간복잡도는 $O(nW)$, 공간복잡도는 $O(W)$이다.

0/1 배낭에서 물건 i를 담을 수 있는 조건은 현재 용량 $c \ge w_i$인 것이다. 물건을 담으면 남은 용량은 $c - w_i$가 된다. 초기 조건은 물건을 하나도 고려하지 않을 때 K(c, 0) = 0, 용량이 0일 때 K(0, i) = 0이다.

optimal BST:

$$\mathrm{OPT}(i,j) = F(i,j) + \min_{i \le r \le j}\{\mathrm{OPT}(i,r-1) + \mathrm{OPT}(r+1,j)\}, \quad O(n^3)$$

F(i, j)는 구간 [i, j]에 속한 키들의 검색 빈도 합이며, 빈 구간에 대해서는 OPT(i, j) = 0 (i > j)으로 둔다.

행렬 연쇄:

$$\mathrm{Cost}(i,j) = \min_{i \le k < j}\{\mathrm{Cost}(i,k) + \mathrm{Cost}(k+1,j) + m_{i-1}m_k m_j\}, \quad O(n^3)$$

행렬 A_i의 크기는 $m_{i-1} \times m_i$이며, Cost(i, i) = 0이다.

pseudo-polynomial	subset sum · knapsack의 $O(nT)$, $O(Wn)$은 **pseudo-polynomial** — T, W의 **값**에 비례 (입력 비트 수엔 지수)
역추적 · 구간 DP	edit distance 역추적 $O(m+n)$ · optimal BST/행렬 연쇄는 구간 (i,j) DP, 대각선(구간 길이) 순서

예시: Edit distance (SNOWY → SUNNY)

	∅	S	U	N	N	Y
∅	0	1	2	3	4	5
S	1	0	1	2	3	4
N	2	1	1	1	2	3
O	3	2	2	2	2	3
W	4	3	3	3	3	3
Y	5	4	4	4	4	3

E(2,3) (SN→SUN) = min(위 2+1, 왼쪽 1+1, 대각선 1+0) = 1. 답 3 — 대각선 역추적: Y=Y 통과, W→N, O→N, N→U 치환, S=S 통과

자주 틀리는 것

부분문제 정의	부분문제는 **"A[i]로 끝나는"**처럼 끝점 고정. max subarray의 답은 $\max_i S(i)$ (마지막 칸이 아님)
LIS 후보 조건	$j < i$와 $A[j] < A[i]$ 둘 다
knapsack	unlimited는 1차원 $K(w)$, 0/1은 2차원 $K(w, j)$